

Property Based Testing

to Preserve Invariants

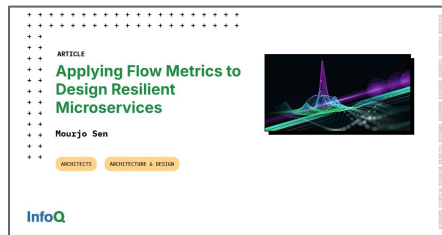
Mourjo Sen

June 3, 2025

<https://mourjo.me>

Namaste!

- Born and brought up in India
- 10 years as a software engineer
- Writing and speaking for 5 years
- Currently employed at Booking.com
- Living in the Netherlands since 2023



Unlocking observability: Structured logging in Spring ...

10 min read · Mar 26, 2024 · [View story](#)

13.6K
Views

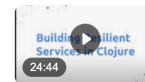
Load Shedding in Web Services

10 min read · Sep 4, 2020 · [View story](#)

13.3K
Views

www.youtube.com › watch

Building Resilient Services in Clojure



Abstract Building resilient web services in Clojure. By employing strategies like circuit breakers, load shedding, graceful restarts among ...

YouTube · reclojure · 21 déc. 2020



Writing a fair task execution framework — Mourjo Sen

YouTube · IN/Clojure
4 févr. 2019

The Invariant That Saved Us

- A few months after moving to Netherlands
- First ever trip to London

The Invariant That Saved Us

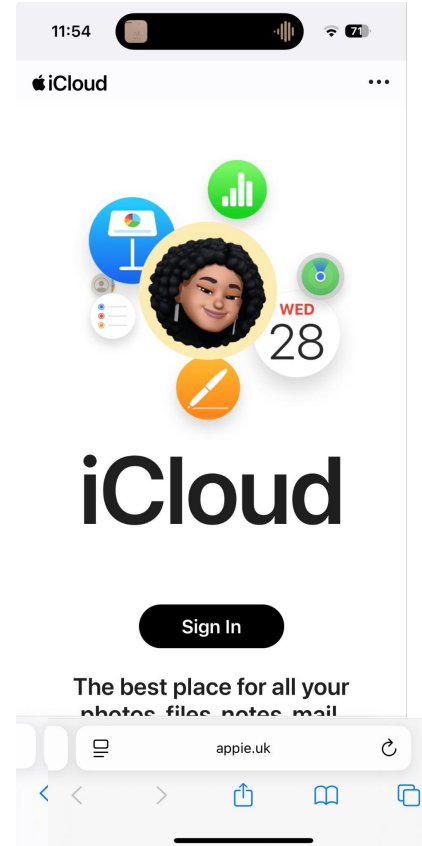
- A few months after moving to Netherlands
- First ever trip to London
- My wife's purse gets stolen

The Invariant That Saved Us

Dear Customer,
Your iPhone 14 128GB Blue has been
found.
View location: <https://apple.uk/38>

The Invariant That Saved Us

Dear Customer,
Your iPhone 14 128GB Blue has been
found.
View location: <https://apple.uk/38>



The Invariant That Saved Us

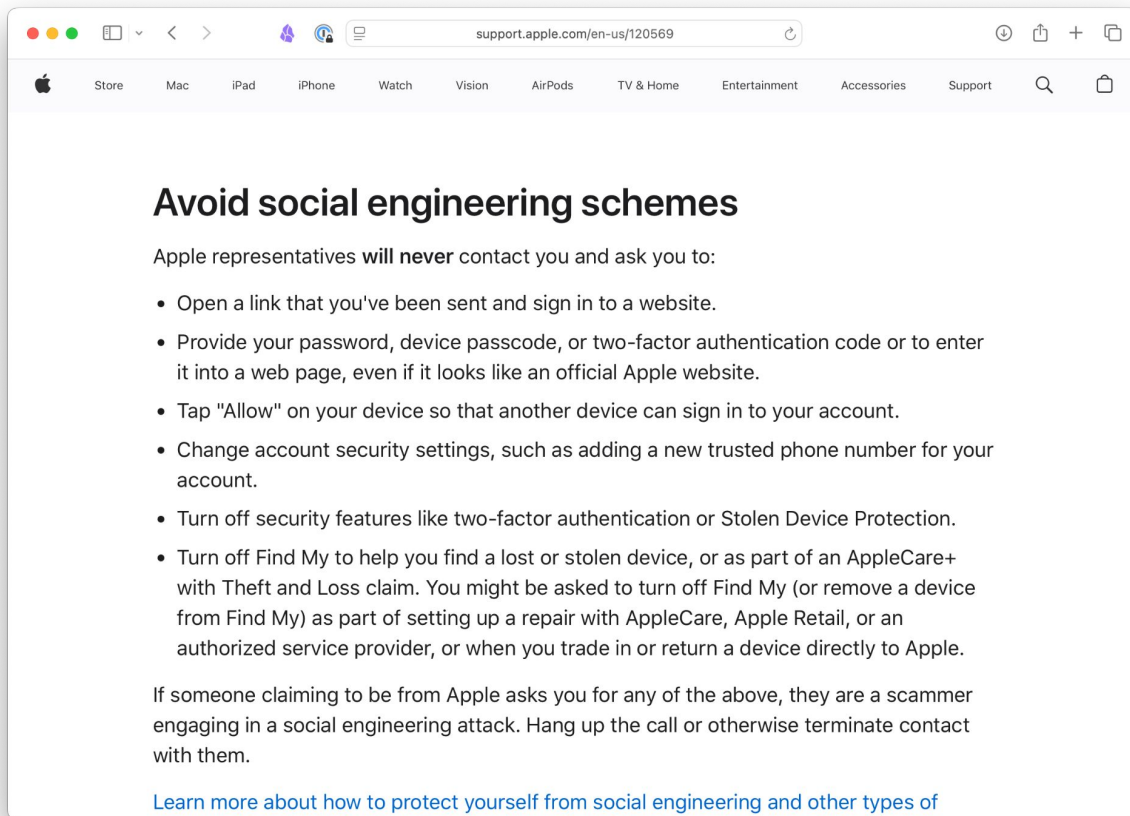
Dear Customer,
Your iPhone 14 128GB Blue has been
found.
View location: <https://apple.uk/38>



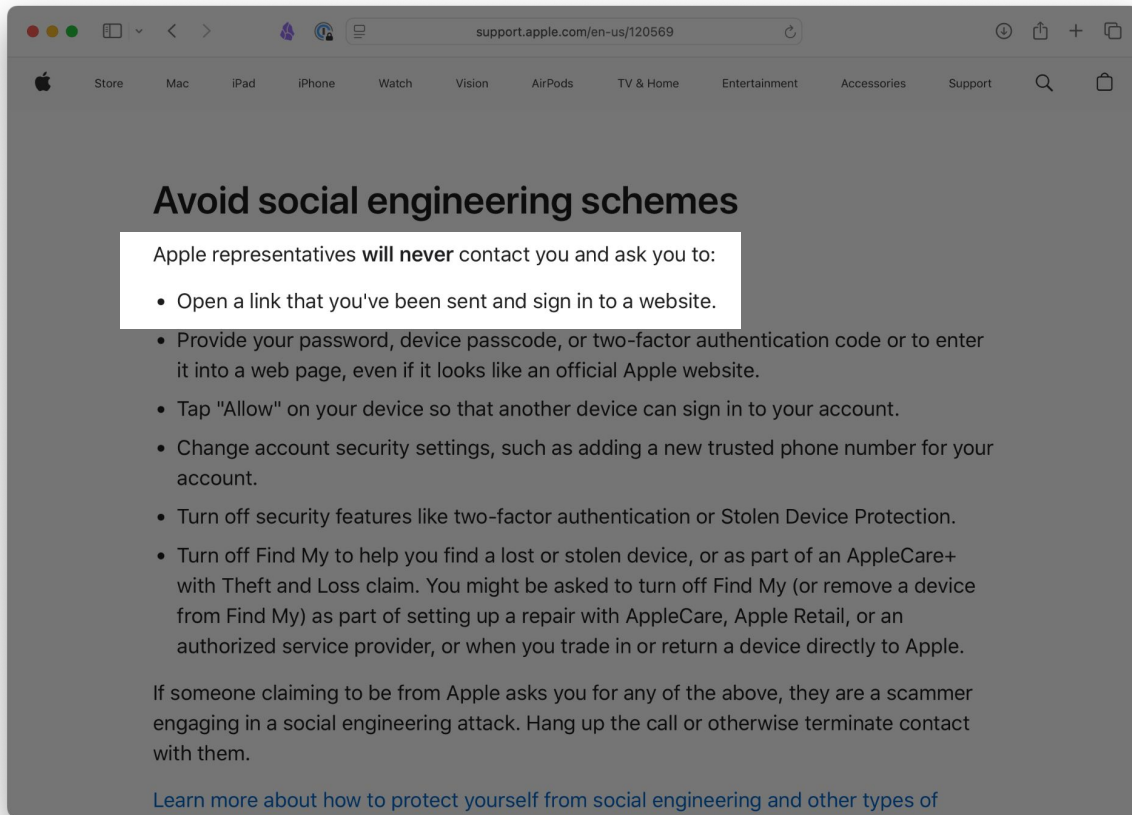
apple.uk



The Invariant That Saved Us

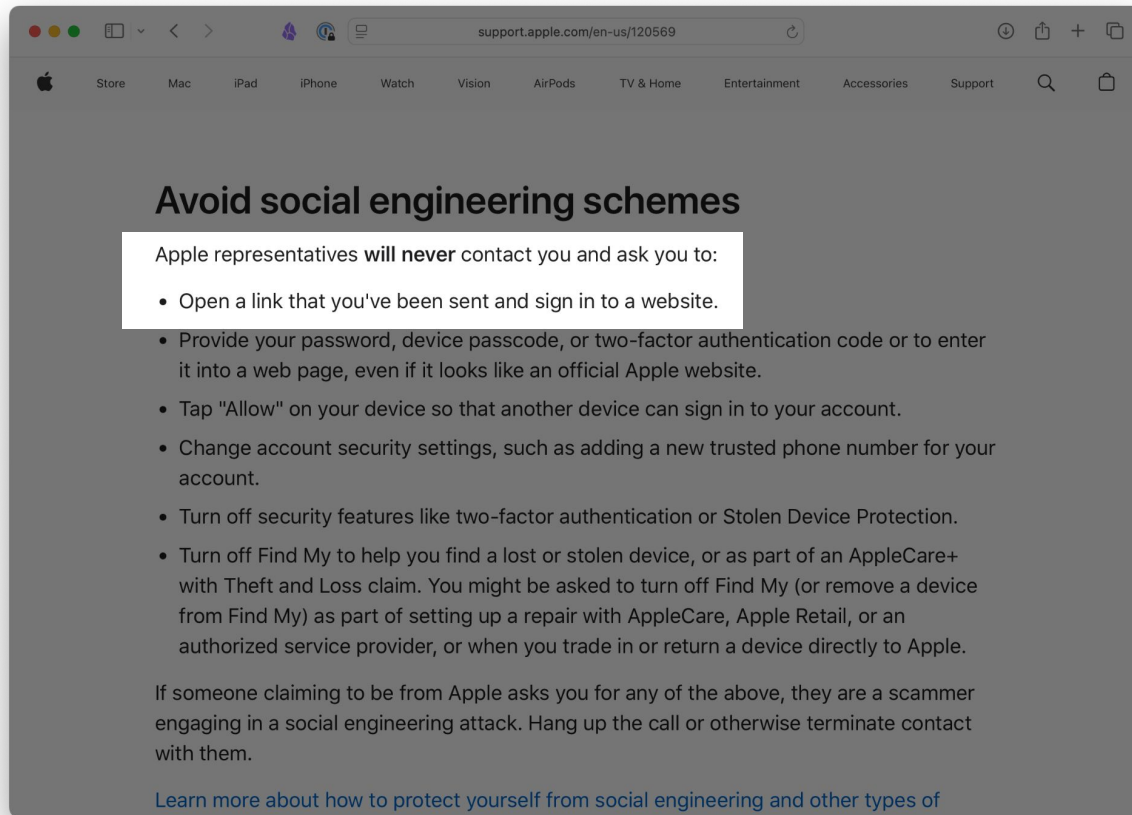


The Invariant That Saved Us



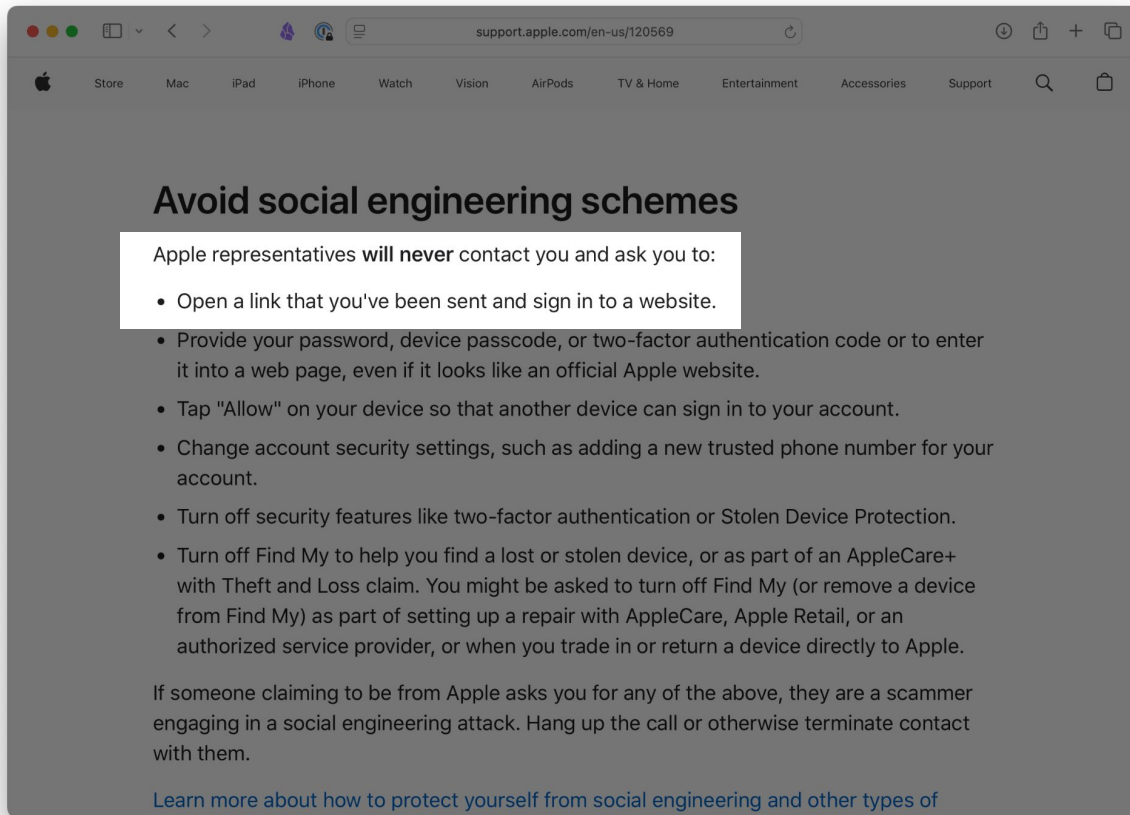
The Invariant That Saved Us

An invariant is a property



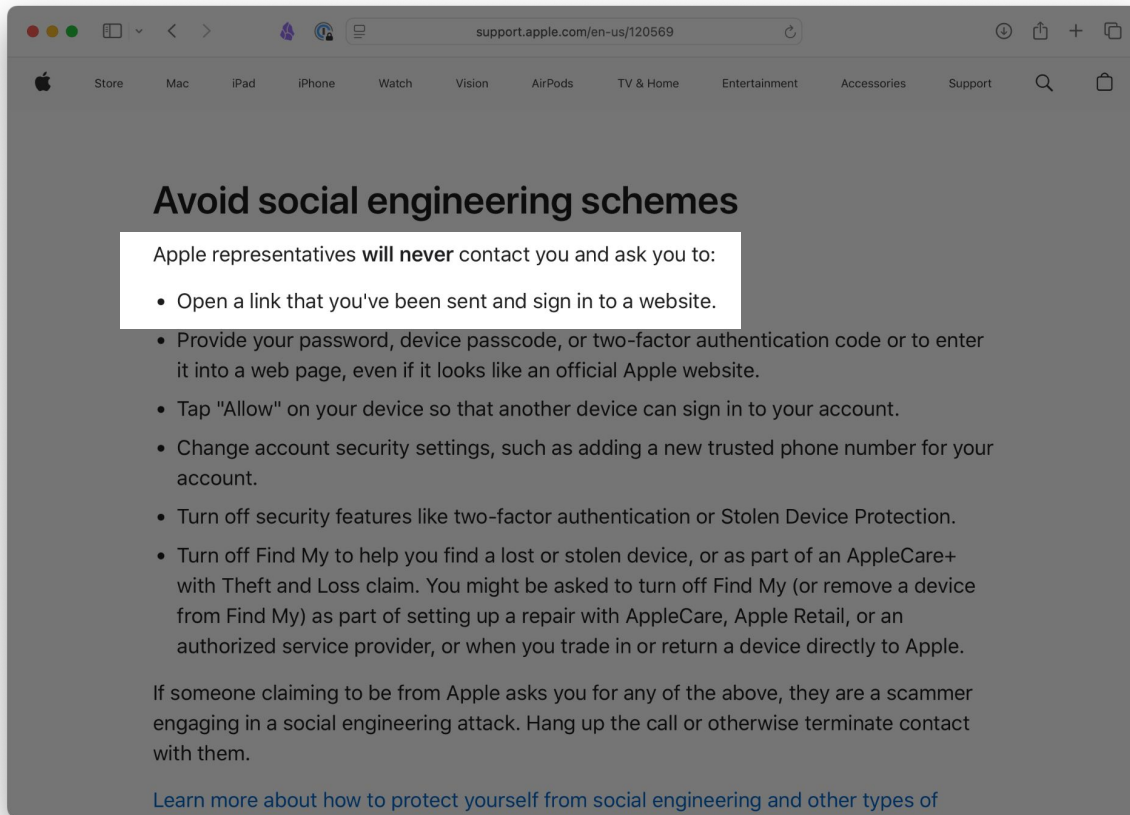
The Invariant That Saved Us

An invariant is a property that remains unchanged



The Invariant That Saved Us

An invariant is a property that remains **unchanged** despite transformations, operations, or processes being applied.



Invariants in Systems

Change is Constant

Change is Constant

- Requirements change

Change is Constant

- Requirements change
- Developers change

Change is Constant

- Requirements change
- Developers change
- Tech stack changes

Change is Constant

- Requirements change
- Developers change
- Tech stack changes
- Infrastructure changes

Change is Constant

- Requirements change
- Developers change
- Tech stack changes
- Infrastructure changes
- Usage patterns change

Change is Constant

- Requirements change
- Developers change
- Tech stack changes
- Infrastructure changes
- Usage patterns change
- **Expectations don't change**

Invariants: Unchanging Properties

- Properties which remain unchanged

Invariants: Unchanging Properties

- Properties which remain unchanged
- Irrespective of any
 - User action

Invariants: Unchanging Properties

- Properties which remain unchanged
- Irrespective of any
 - User action
 - System state

Invariants: Unchanging Properties

- Properties which remain unchanged
- Irrespective of any
 - User action
 - System state
 - Code change
 - ...

Invariants: Unchanging Properties

- The user never receives stack traces

Invariants: Unchanging Properties

- The user never receives stack traces
- Amount of money is unchanged after an account transfer

Invariants: Unchanging Properties

- The user never receives stack traces
- Amount of money is unchanged after an account transfer
- One person cannot be in two meetings at the same time

Invariants: Unchanging Properties

- The user never receives stack traces
- Amount of money is unchanged after an account transfer
- One person cannot be in two meetings at the same time
- Every element in a sorted array $\leq$ the next element
- ...

Property Based Testing

- Given a **property** (aka invariant)

Property Based Testing

- Given a **property** (aka **invariant**)
- Generates **random** inputs

Property Based Testing

- Given a **property** (aka **invariant**)
- Generates **random** inputs
- Finds the **smallest input** for which the property **does not hold**

A Leap Year Example

```
@Test
void exampleTests() {
    assertThat(app.lastDayOfFeb(2015)).isEqualTo(28);
    assertThat(app.lastDayOfFeb(2020)).isEqualTo(29);
    assertThat(app.lastDayOfFeb(2021)).isEqualTo(28);
}
```


A Leap Year Example

```
@Test
void exampleTests() {
    assertThat(app.lastDayOfFeb(2015)).isEqualTo(28);
    assertThat(app.lastDayOfFeb(2020)).isEqualTo(29);
    assertThat(app.lastDayOfFeb(2021)).isEqualTo(28);
}
```

A Leap Year Example

@Test

```
 void exampleTests() {  
    assertThat(app.lastDayOfFeb(2015)).isEqualTo(28);  
    assertThat(app.lastDayOfFeb(2020)).isEqualTo(29);  
    assertThat(app.lastDayOfFeb(2021)).isEqualTo(28);  
}
```

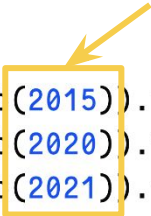
@Property

```
void propTests(@ForAll @IntRange(min = 1800) int year) {  
    int calendarDate = LocalDateTime.of(year, 3, 1, 0, 0, 0).minusDays(1).getDayOfMonth();  
    assertThat(app.lastDayOfFeb(year)).isEqualTo(calendarDate);  
}
```

A Leap Year Example

Hand-crafted list of years

```
@Test
void exampleTests() {
    assertThat(app.lastDayOfFeb(2015)).isEqualTo(28);
    assertThat(app.lastDayOfFeb(2020)).isEqualTo(29);
    assertThat(app.lastDayOfFeb(2021)).isEqualTo(28);
}
```



```
@Property
void propTests(@ForAll @IntRange(min = 1800) int year) {
    int calendarDate = LocalDateTime.of(year, 3, 1, 0, 0, 0).minusDays(1).getDayOfMonth();
    assertThat(app.lastDayOfFeb(year)).isEqualTo(calendarDate);
}
```

A Leap Year Example

Hand-crafted list of years

```
@Test
void exampleTests() {
    assertThat(app.lastDayOfFeb(2015)).isEqualTo(28);
    assertThat(app.lastDayOfFeb(2020)).isEqualTo(29);
    assertThat(app.lastDayOfFeb(2021)).isEqualTo(28);
}
```

Every year since 1800

```
@Property
void propTests(@ForAll @IntRange(min = 1800) int year) {
    int calendarDate = LocalDateTime.of(year, 3, 1, 0, 0, 0).minusDays(1).getDayOfMonth();
    assertThat(app.lastDayOfFeb(year)).isEqualTo(calendarDate);
}
```

A Leap Year Example

Hand-crafted list of years

```
@Test
void exampleTests() {
    assertThat(app.lastDayOfFeb(2015)).isEqualTo(28);
    assertThat(app.lastDayOfFeb(2020)).isEqualTo(29);
    assertThat(app.lastDayOfFeb(2021)).isEqualTo(28);
}
```

Every year since 1800

```
@Property
void propTests(@ForAll @IntRange(min = 1800) int year) {
    int calendarDate = LocalDateTime.of(year, 3, 1, 0, 0, 0).minusDays(1).getDayOfMonth();
    assertThat(app.lastDayOfFeb(year)).isEqualTo(calendarDate);
}
```

A Leap Year Example

```
@Test
void example() {
    assertEquals(1, 1)
    assertEquals(2, 2)
    assertEquals(3, 3)
}

@Property
void property() {
    int value = 1
    assertEquals(1, value)
}
```

```
tries = 51
checks = 51
generation = EXHAUSTIVE
after-failure = RANDOM_SEED
when-fixed-seed = ALLOW
edge-cases#mode = MIXIN
edge-cases#total = 0
edge-cases#tried = 0
seed = 6038683740369428047
```

-----jqwik-----
of calls to property
of not rejected calls
parameters are exhaustively generated
use a new random seed
fixing the random seed is allowed
edge cases are mixed in
of all combined edge cases
of edge cases tried in current run
random seed to reproduce generated values

Sample

year: 1900

.DayOfMonth();

A Leap Year Example

```
@Test
void example() {
    assertEquals(1900, isLeapYear(1900));
    assertEquals(1904, isLeapYear(1904));
    assertEquals(1908, isLeapYear(1908));
}

@Property
void properties() {
    int calls = 0;
    assertEquals(51, calls);
}
```

```
tries = 51
checks = 51
generation = EXHAUSTIVE
after-failure = RANDOM_SEED
when-fixed-seed = ALLOW
edge-cases#mode = MIXIN
edge-cases#total = 0
edge-cases#tried = 0
seed = 6038683740369428047
```

```
|-----jqwik-----|
| # of calls to property
| # of not rejected calls
| parameters are exhaustively generated
| use a new random seed
| fixing the random seed is allowed
| edge cases are mixed in
| # of all combined edge cases
| # of edge cases tried in current run
| random seed to reproduce generated values
```

Sample

year: 1900

.DayOfMonth();

A Leap Year Example

@Test

```
 void exampleTests() {  
    assertThat(app.lastDayOfFeb(2015)).isEqualTo(28);  
    assertThat(app.lastDayOfFeb(2020)).isEqualTo(29);  
    assertThat(app.lastDayOfFeb(2021)).isEqualTo(28);  
}
```

@Property

```
 void propTests(@ForAll @IntRange(min = 1800) int year) {  
    int calendarDate = LocalDateTime.of(year, 3, 1, 0, 0, 0).minusDays(1).getDayOfMonth();  
    assertThat(app.lastDayOfFeb(year)).isEqualTo(calendarDate);  
}
```


A Leap Year Example

```
@Test
void exampleTests() {
    assertThat(app.lastDayOfFeb(2015)).isEqualTo(28);
    assertThat(app.lastDayOfFeb(2020)).isEqualTo(29);
    assertThat(app.lastDayOfFeb(2021)).isEqualTo(28);
}
```

Sampled test-cases are
prone to false positives

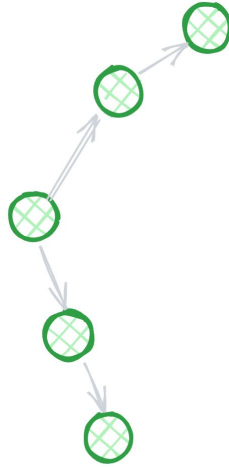
```
@Property
void propTests(@ForAll @IntRange(min = 1800) int year) {
    int calendarDate = LocalDateTime.of(year, 3, 1, 0, 0, 0).minusDays(1).getDayOfMonth();
    assertThat(app.lastDayOfFeb(year)).isEqualTo(calendarDate);
}
```

A Leap Year Example

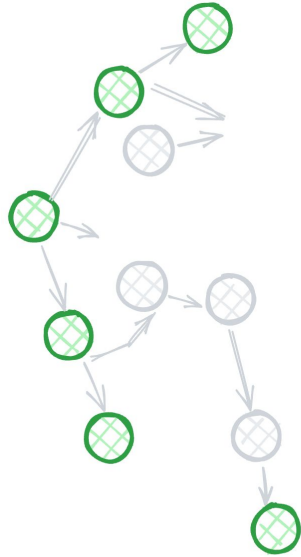


Sampled test-cases are
prone to false positives

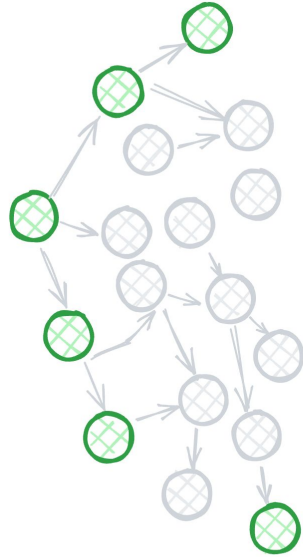
Sampled Test Cases: Accidental Quality



Sampled Test Cases: Accidental Quality



Sampled Test Cases: Accidental Quality



Sampled Test Cases: Accidental Quality



Sampled Test Cases: Accidental Quality



Sampled Test Cases: Accidental Quality

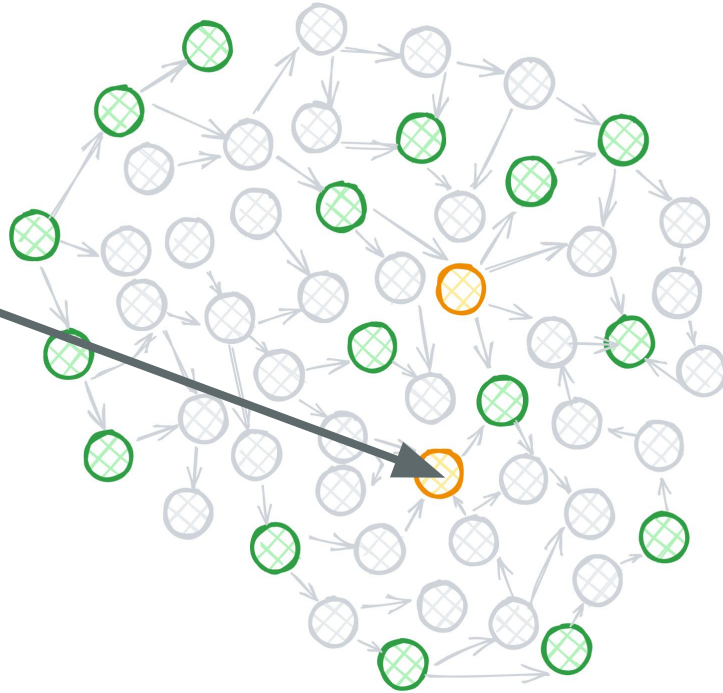


Sampled Test Cases: Accidental Quality

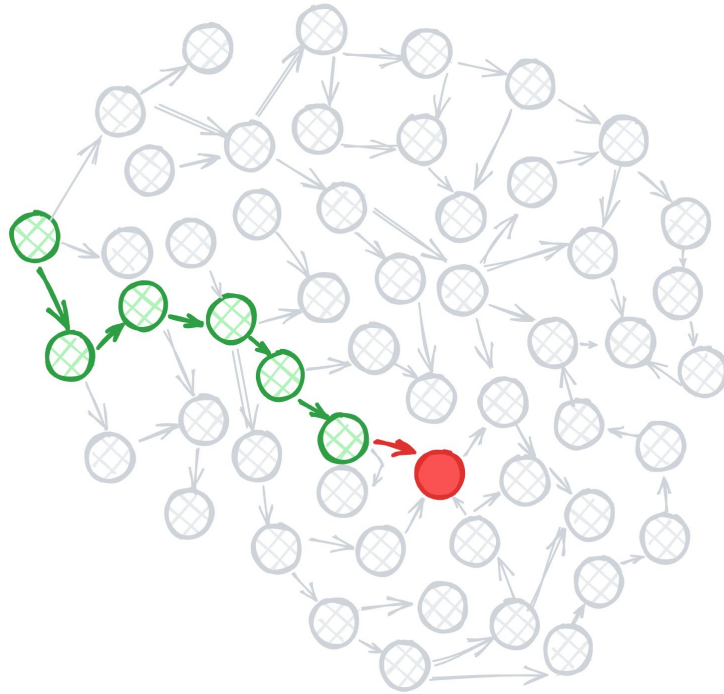


Sampled Test Cases: Accidental Quality

Write a test if
we *think* it will
cause a bug

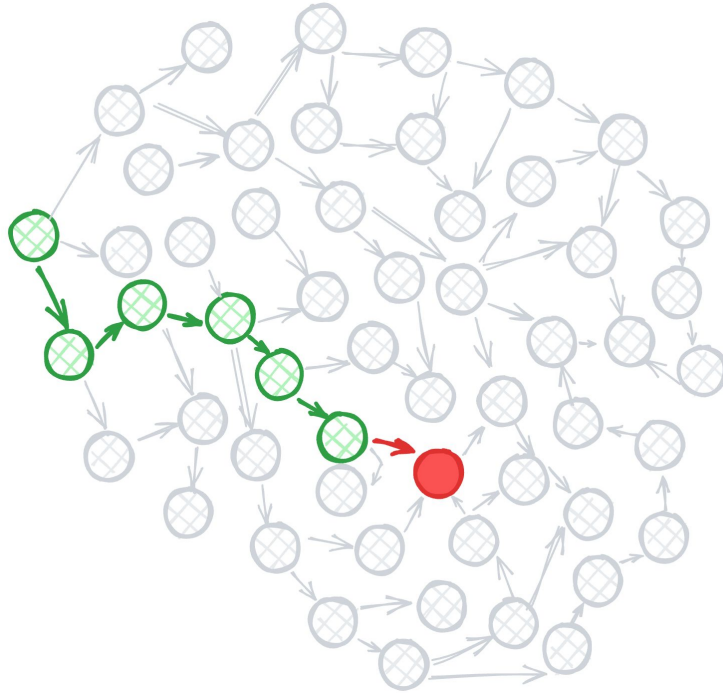


Search and Not Just Test For Bugs



Search and Not Just Test For Bugs

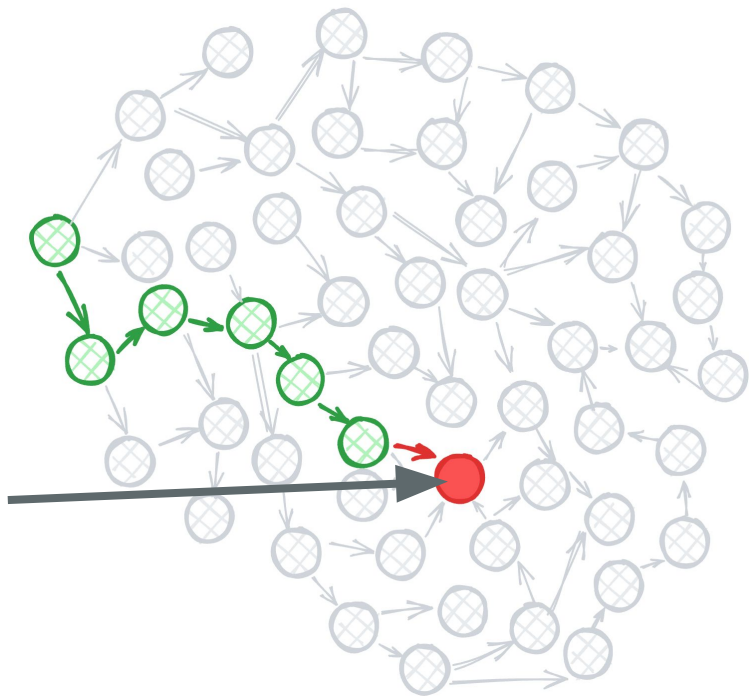
Without prior hint
of bug existence



Search and Not Just Test For Bugs

Without prior hint
of bug existence

What minimally
triggers this bug?



Search and Not Just Test For Bugs

```
@Test
void exampleTests() {
    assertThat(app.bubbleSort(List.of(10, 20, 30))).isEqualTo(List.of(10, 20, 30));
    assertThat(app.bubbleSort(List.of(5, 2, 6))).isEqualTo(List.of(2, 5, 6));
    assertThat(app.bubbleSort(List.of(100, -200, 101))).isEqualTo(List.of(-200, 100, 101));
    assertThat(app.bubbleSort(List.of(100, 80, 530))).isEqualTo(List.of(80, 100, 530));
}
```

Search and Not Just Test For Bugs

Sampled inputs

```
@Test
void exampleTests() {
    assertThat(app.bubbleSort(List.of(10, 20, 30))).isEqualTo(List.of(10, 20, 30));
    assertThat(app.bubbleSort(List.of(5, 2, 6))).isEqualTo(List.of(2, 5, 6));
    assertThat(app.bubbleSort(List.of(100, -200, 101))).isEqualTo(List.of(-200, 100, 101));
    assertThat(app.bubbleSort(List.of(100, 80, 530))).isEqualTo(List.of(80, 100, 530));
}
```



Search and Not Just Test For Bugs

Sampled inputs

```
@Test
void exampleTests() {
    assertThat(app.bubbleSort(List.of(10, 20, 30))).isEqualTo(List.of(10, 20, 30));
    assertThat(app.bubbleSort(List.of(5, 2, 6))).isEqualTo(List.of(2, 5, 6));
    assertThat(app.bubbleSort(List.of(100, -200, 101))).isEqualTo(List.of(-200, 100, 101));
    assertThat(app.bubbleSort(List.of(100, 80, 530))).isEqualTo(List.of(80, 100, 530));
}

@Property(afterFailure = AfterFailureMode.RANDOM_SEED)
void propTests(@ForAll List<Integer> integers) {
    var sortedIntegers = app.bubbleSort(integers);
    for (int i = 0; i < integers.size() - 1; i++) {
        assertThat(sortedIntegers.get(i)).isLessThanOrEqualTo(sortedIntegers.get(i + 1));
    }
}
```


Search and Not Just Test For Bugs

Sampled inputs

```
@Test
void exampleTests() {
    assertThat(app.bubbleSort(List.of(10, 20, 30))).isEqualTo(List.of(10, 20, 30));
    assertThat(app.bubbleSort(List.of(5, 2, 6))).isEqualTo(List.of(2, 5, 6));
    assertThat(app.bubbleSort(List.of(100, -200, 101))).isEqualTo(List.of(-200, 100, 101));
    assertThat(app.bubbleSort(List.of(100, 80, 530))).isEqualTo(List.of(80, 100, 530));
}
```

All possible arrays of integers

```
@Property(afterFailure = AfterFailureMode.RANDOM_SEED)
void propTests(@ForAll List<Integer> integers) {
    var sortedIntegers = app.bubbleSort(integers);
    for (int i = 0; i < integers.size() - 1; i++) {
        assertThat(sortedIntegers.get(i)).isLessThanOrEqualTo(sortedIntegers.get(i + 1));
    }
}
```

Search and Not Just Test For Bugs

Sampled inputs

```
@Test
void exampleTests() {
    assertThat(app.bubbleSort(List.of(10, 20, 30))).isEqualTo(List.of(10, 20, 30));
    assertThat(app.bubbleSort(List.of(5, 2, 6))).isEqualTo(List.of(2, 5, 6));
    assertThat(app.bubbleSort(List.of(100, -200, 101))).isEqualTo(List.of(-200, 100, 101));
    assertThat(app.bubbleSort(List.of(100, 80, 530))).isEqualTo(List.of(80, 100, 530));
}
```

All possible arrays of integers

```
@Property(afterFailure = AfterFailureMode.RANDOM_SEED)
void propTests(@ForAll List<Integer> integers) {
    var sortedIntegers = app.bubbleSort(integers);
    for (int i = 0; i < integers.size() - 1; i++) {
        assertThat(sortedIntegers.get(i)).isLessThanOrEqualTo(sortedIntegers.get(i + 1));
    }
}
```

Search and Not Just Test For Bugs

Sampled inputs

```
@Test
void exampleTests() {
    assertThat(app.bubbleSort(List.of(10, 20, 30))).isEqualTo(List.of(10, 20, 30));
    assertThat(app.bubbleSort(List.of(5, 2, 6))).isEqualTo(List.of(2, 5, 6));
    assertThat(app.bubbleSort(List.of(100, -200, 101))).isEqualTo(List.of(-200, 100, 101));
    assertThat(app.bubbleSort(List.of(100, 80, 530))).isEqualTo(List.of(80, 100, 530));
}
```

All possible arrays of integers

```
@Property(afterFailure = AfterFailureMode.RANDOM_SEED)
void propTests(@ForAll List<Integer> integers) {
    var sortedIntegers = app.bubbleSort(integers);
    for (int i = 0; i < integers.size() - 1; i++) {
        assertThat(sortedIntegers.get(i)).isLessThanOrEqualTo(sortedIntegers.get(i + 1));
    }
}
```

Search and Not Just Test For Bugs

Original Sample

integers:

[

-644, -962072, 4372, -770, -1378, -166689, -1005137436,
-22, 66, 31, -96, -982992, 14, 889, -13542, -22347,
-887352264, -458103, 1457, 2, -256738, 1492, -1371,
286553, 11002, -147847, -1984867825, -176

]

Search and Not Just Test For Bugs

Shrunk Sample (11 steps)

integers: **[0, -1]**

Original Sample

integers:

[

-644, -962072, 4372, -770, -1378, -166689, -1005137436,
-22, 66, 31, -96, -982992, 14, 889, -13542, -22347,
-887352264, -458103, 1457, 2, -256738, 1492, -1371,
286553, 11002, -147847, -1984867825, -176

]

Search and Not Just Test For Bugs

Shrunk Sample (11 steps)

integers: **[1, 0]**

Original Sample

integers:

```
[  
  0, -2, 5, -24, -5888315, 386, 319101, 153, -1530,  
  3496680, -62, -18160  
]
```

Search and Not Just Test For Bugs

Shrunk Sample (11 steps)

integers: **[0, -1]**

Original Sample

integers:

[

2147483647, -1092, -12689, -10521, 2571, 1917, 217,
-132342, 21, 11280, 2544, -154523, 3823, 848716, -211,
607306, -1472, 5147375, 878291, 6, 5587629, 348,
1077451186, 1200, -766077, 176, 51, -1432, 134

]

Search and Not Just Test For Bugs

Shrunk Sample (11 steps)

integers: **[1, 0]**

Original Sample

integers:

[

4839, -468, -17, 2, 574, 36736, -543, -55, -1698,
-2147483648, 148, -4, 1507, -1, 194, -678, -161, 67259,
1, -66, -23, -5356541, 396460, 20, -928, -1

]

Search and Not Just Test For Bugs

Shrunk Sample (11 steps)

integers: **[1, 0]**

Original Sample

integers:

[

-18, 1834, -12651, -32, 6, -532, -17531, -148775, 172,
22159, 568, 13

]

Search and Not Just Test For Bugs

Shrunk Sample (11 steps)

integers: **[1, 0]**

Original Sample

integers:

[

-18, 1834, -12651, -32, 6, -532, -17531, -148775, 172,
22159, 568, 13

]

The last number
never gets sorted

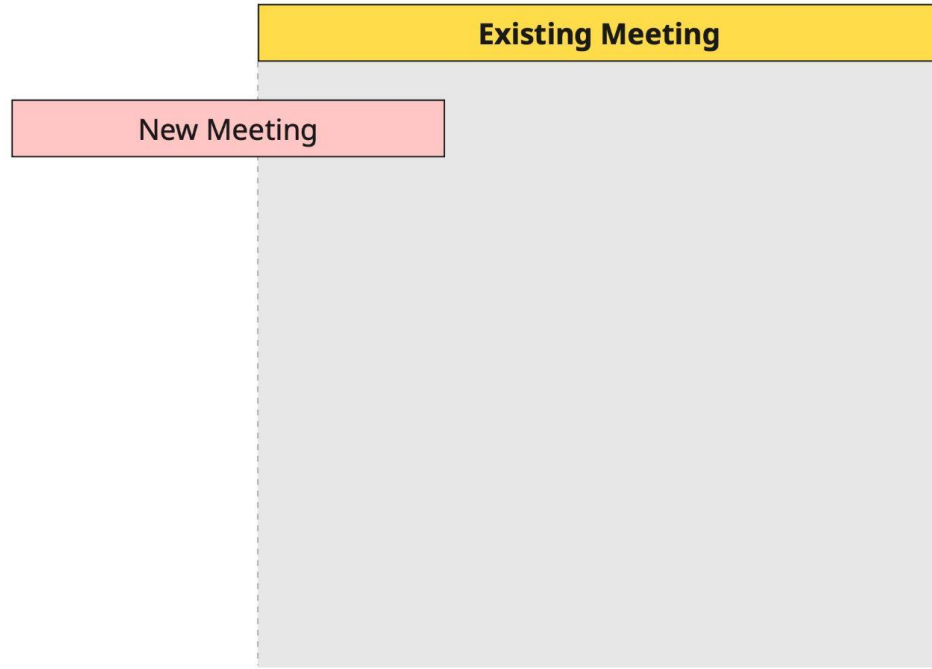


Invariants in A Meetings Application

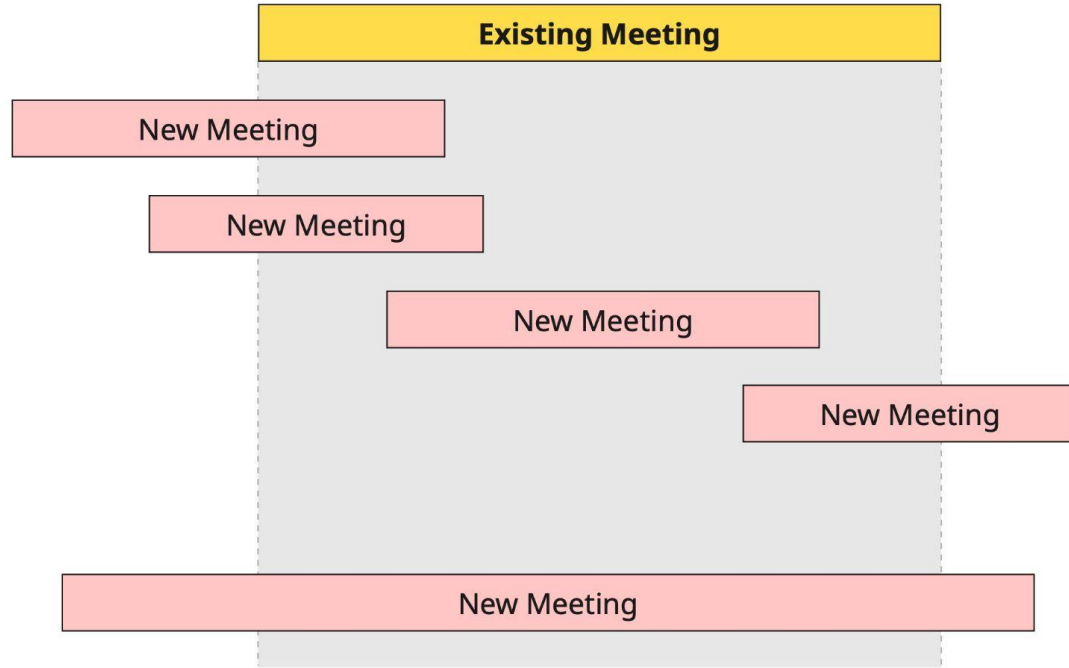
A Meetings Application

Existing Meeting

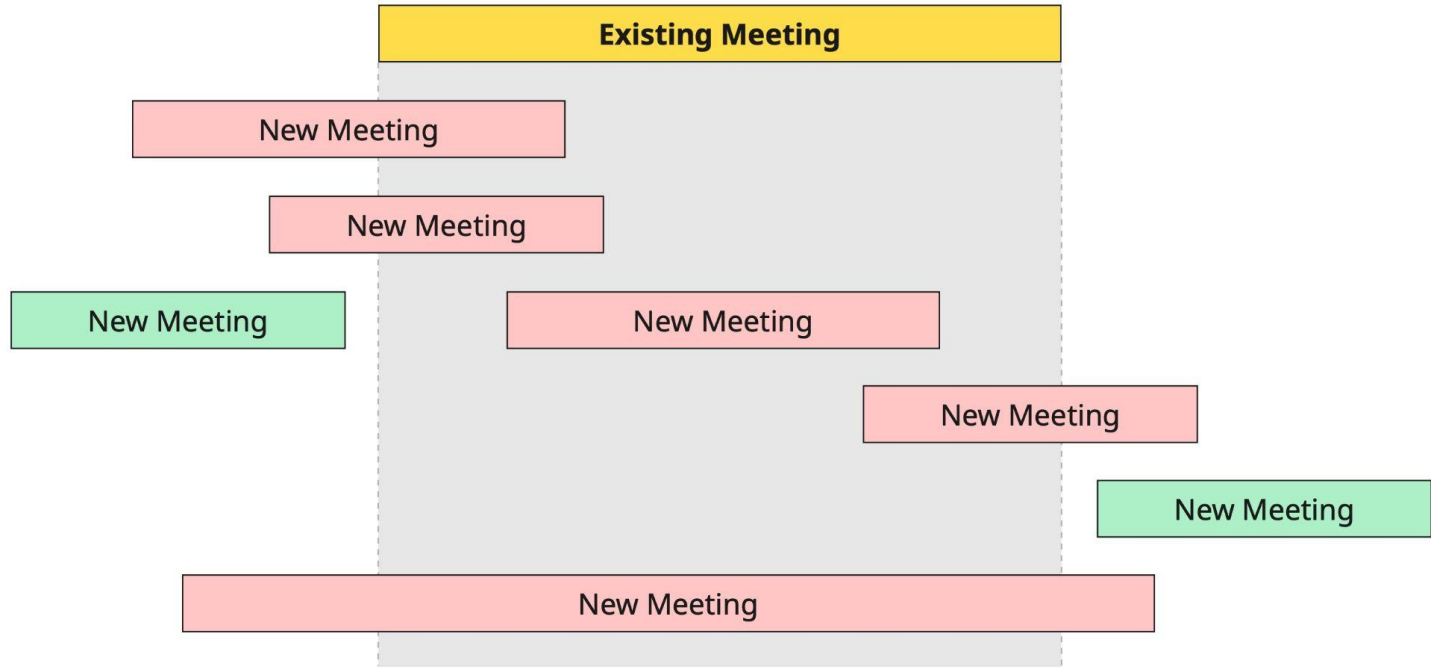
A Meetings Application



A Meetings Application



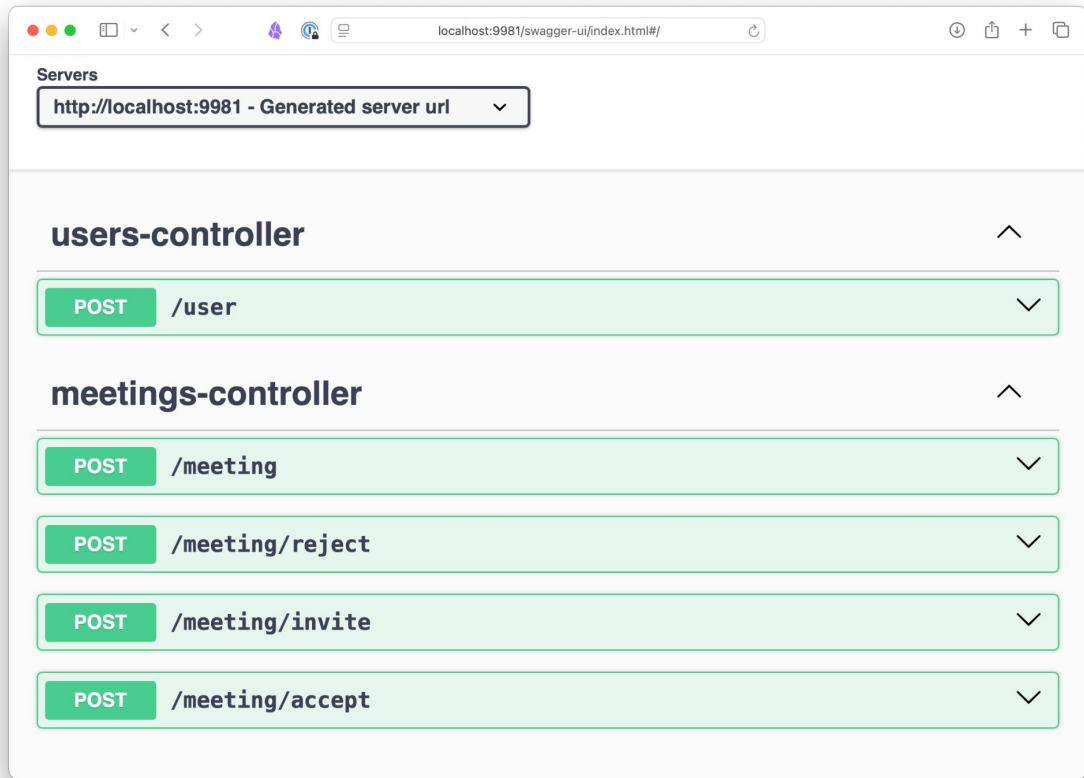
A Meetings Application



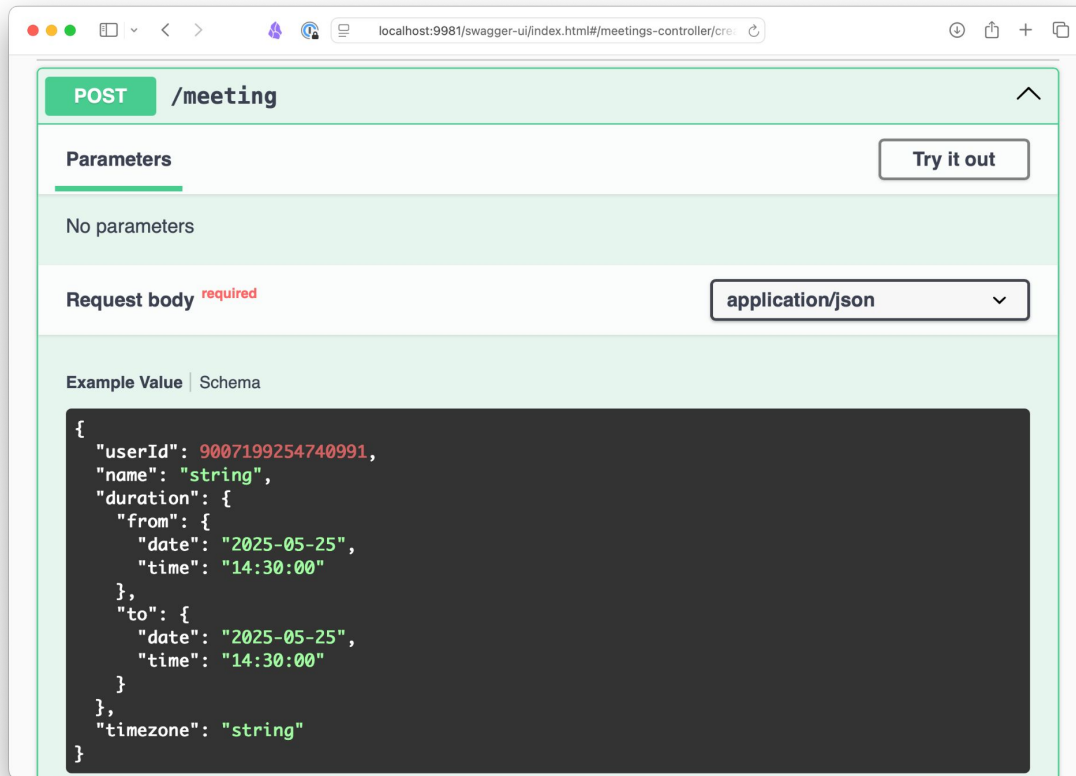
Simple Capabilities

- Create meetings
- Invite others to meetings
- Accept or reject invitations

Simple Capabilities



Simple Capabilities



Simple Capabilities

The image shows a web browser window displaying the Swagger UI for a REST API. The URL in the address bar is `localhost:9981/swagger-ui/index.html#/meetings-controller/create`. The interface is for a **POST** request to the `/meeting` endpoint. It features a 'Parameters' section with 'No parameters' listed, and a 'Request body' section with a red 'required' label. A dropdown menu shows 'application/json' as the selected content type. The request body is a JSON object: `{ "userId": 260893, "name": "Alice's First Meeting", "timezone": "Asia/Kolkata" }`. 'Cancel' and 'Reset' buttons are visible in the top right of the parameters section.

POST /meeting

Parameters

Cancel Reset

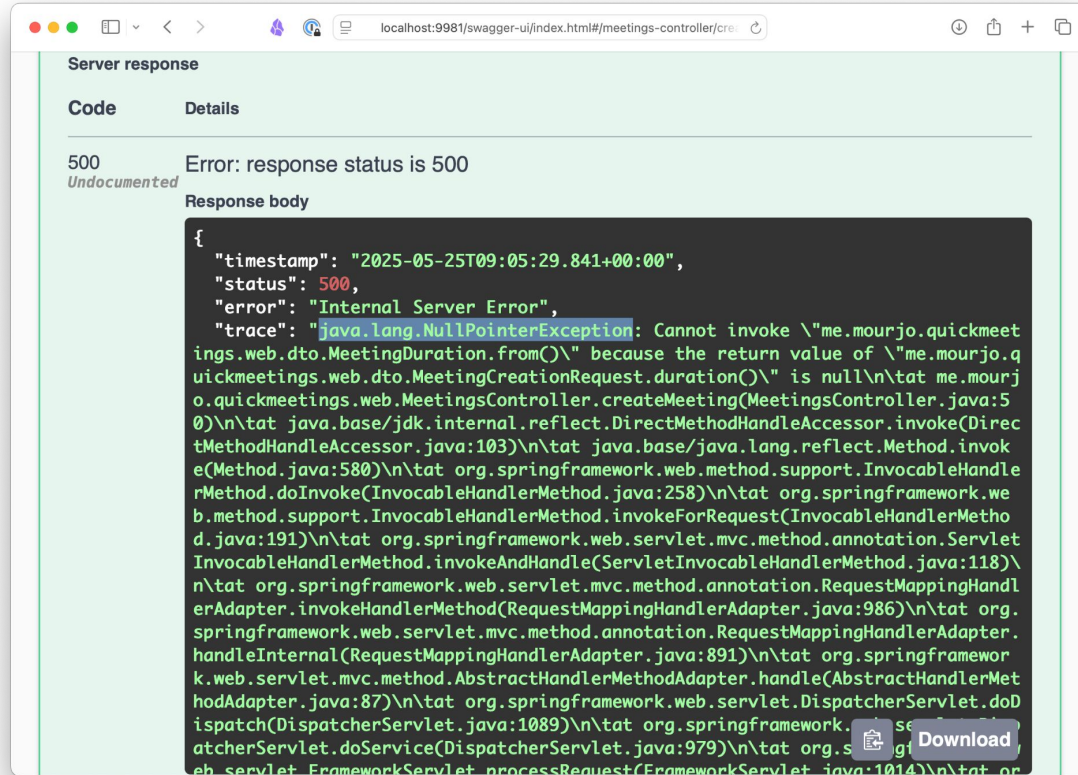
No parameters

Request body **required**

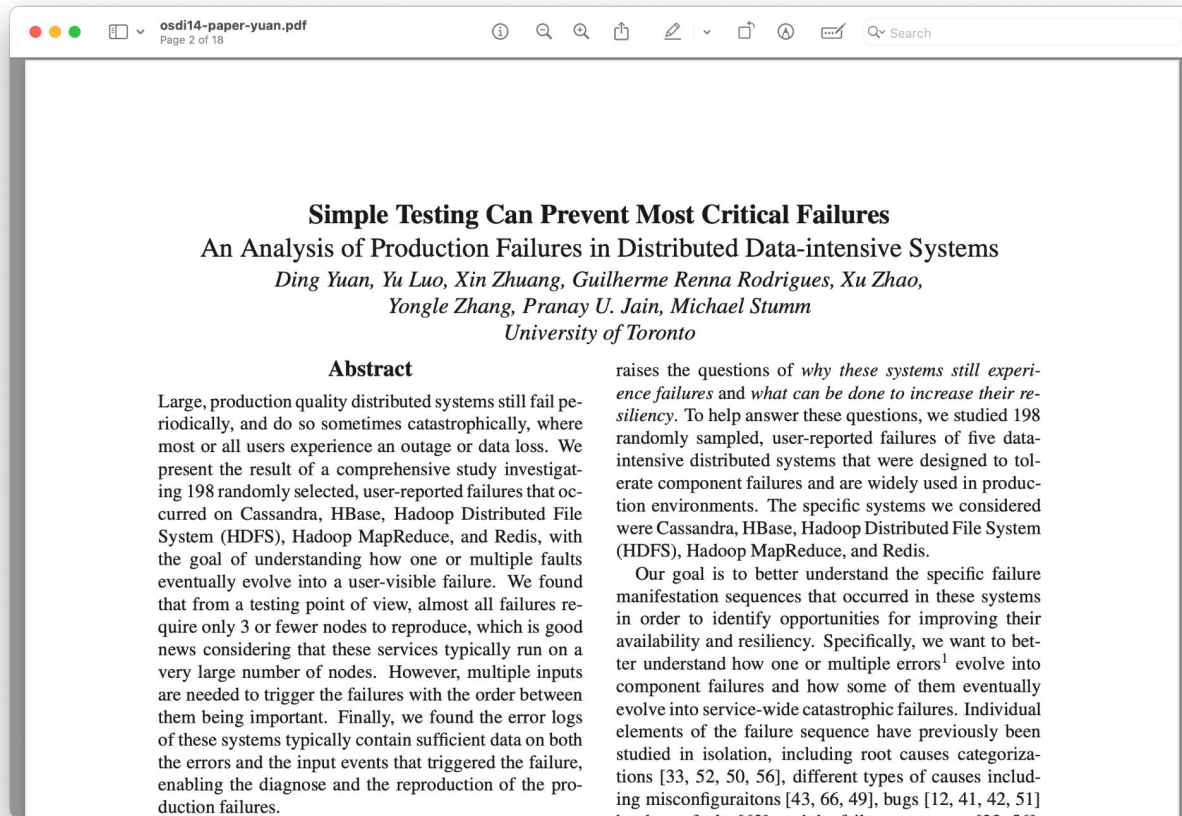
application/json

```
{
  "userId": 260893,
  "name": "Alice's First Meeting",
  "timezone": "Asia/Kolkata"
}
```

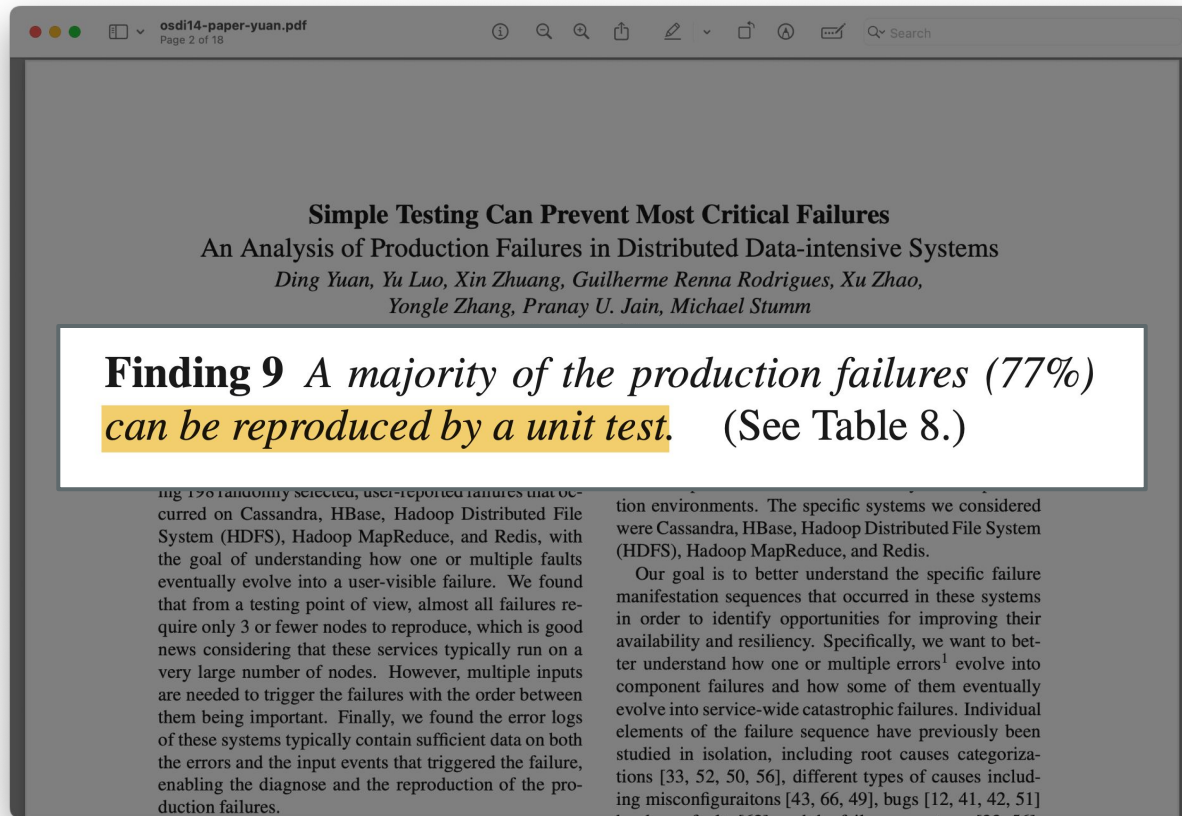
Simple Capabilities: Numerous Fault Lines



Simple Capabilities: Numerous Fault Lines

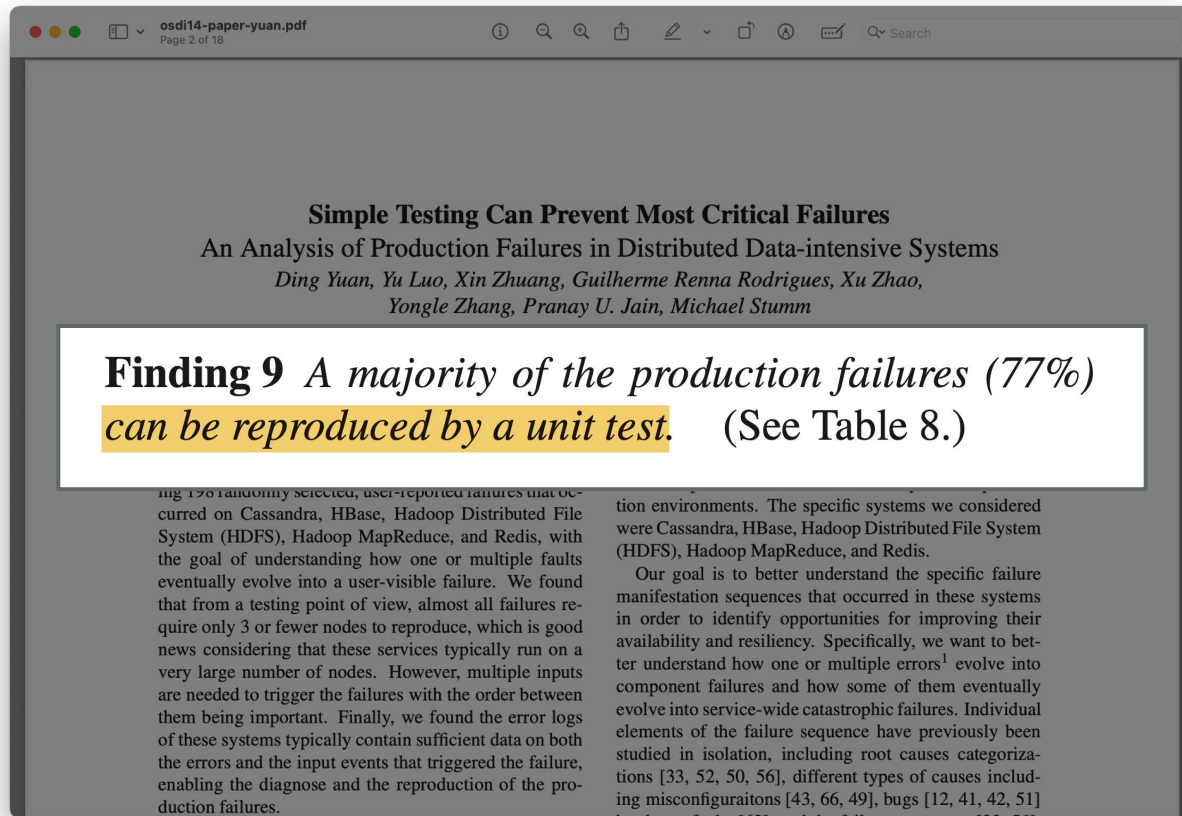


Simple Capabilities: Numerous Fault Lines

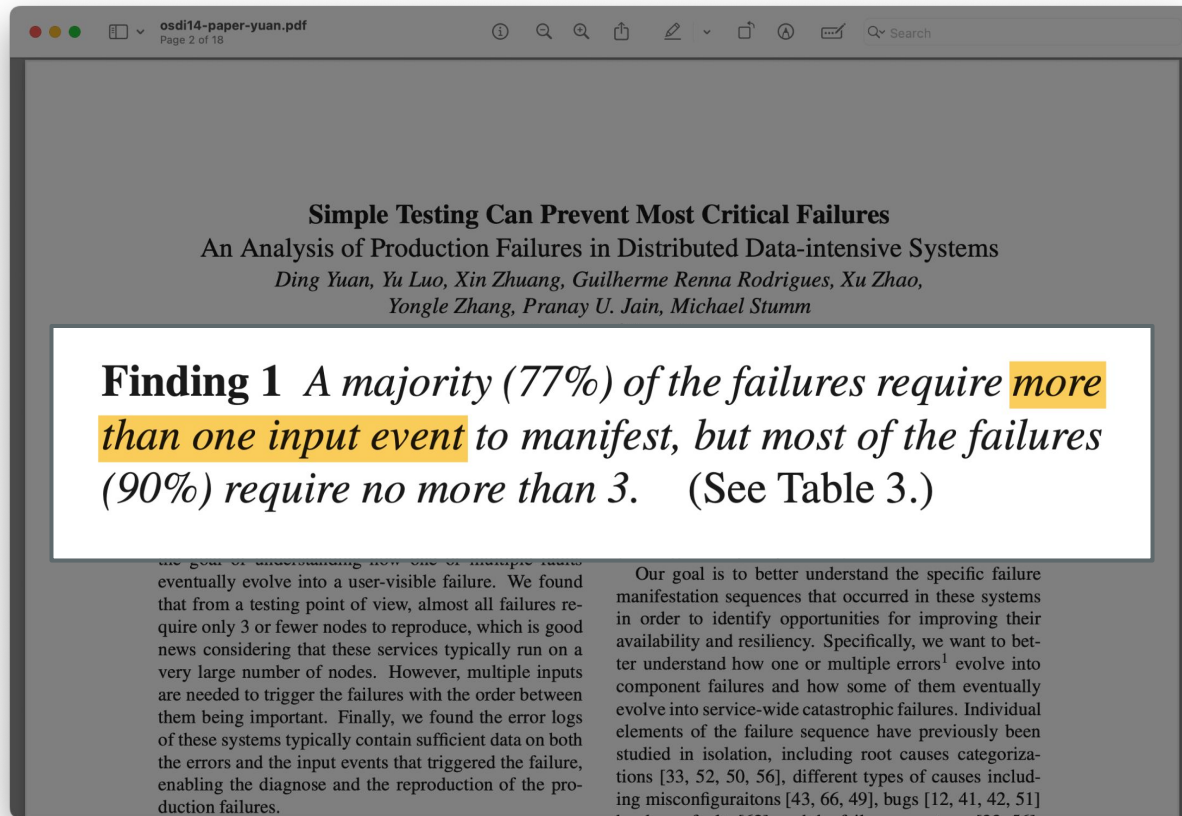


Simple Capabilities: Numerous Fault Lines

Have we
written
enough unit
tests?

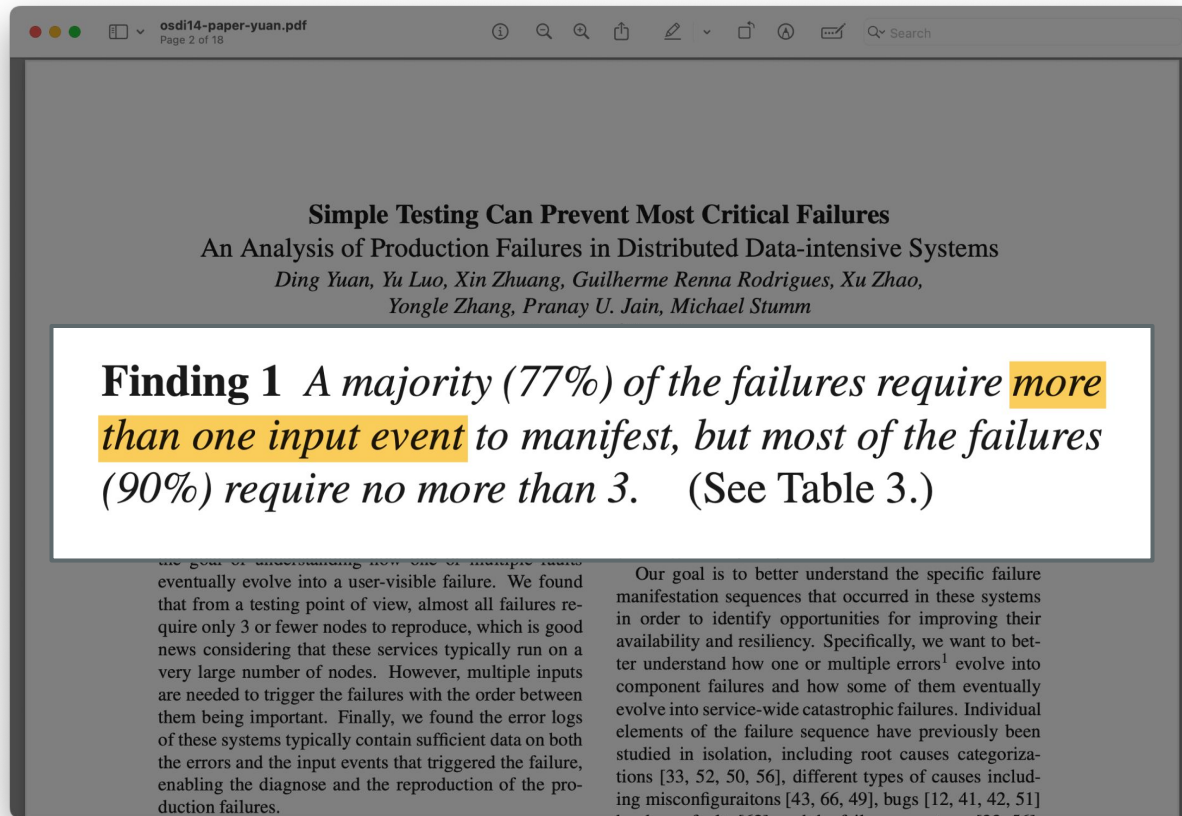


Simple Capabilities: Numerous Fault Lines



Simple Capabilities: Numerous Fault Lines

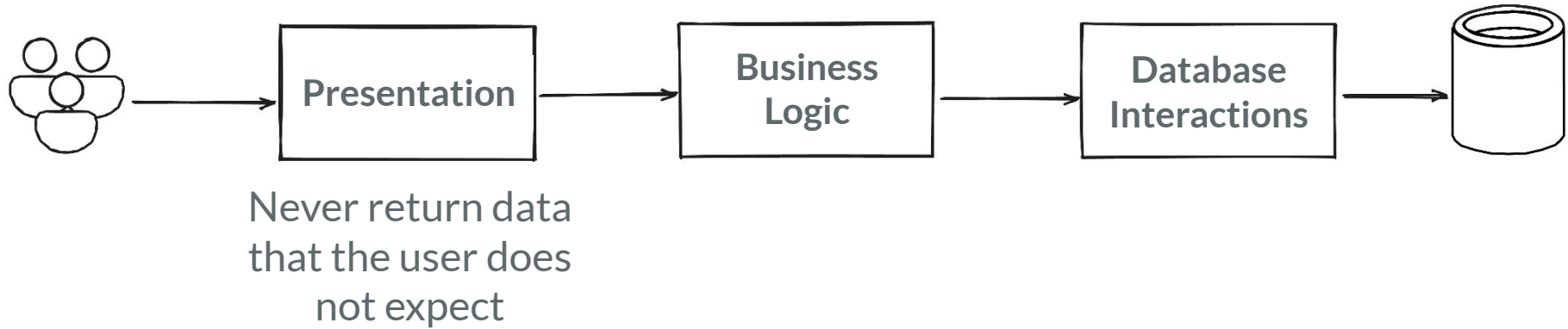
Combinatorial
search
problem



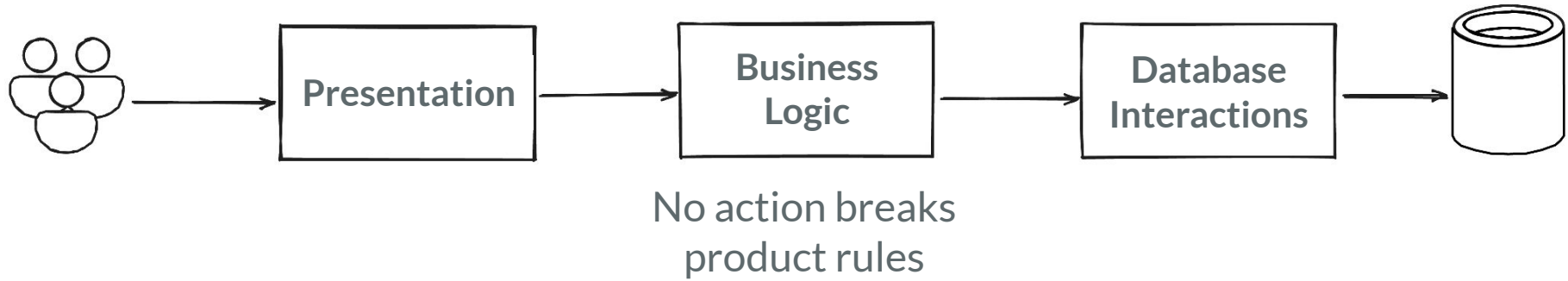
A Search For Lurking Bugs



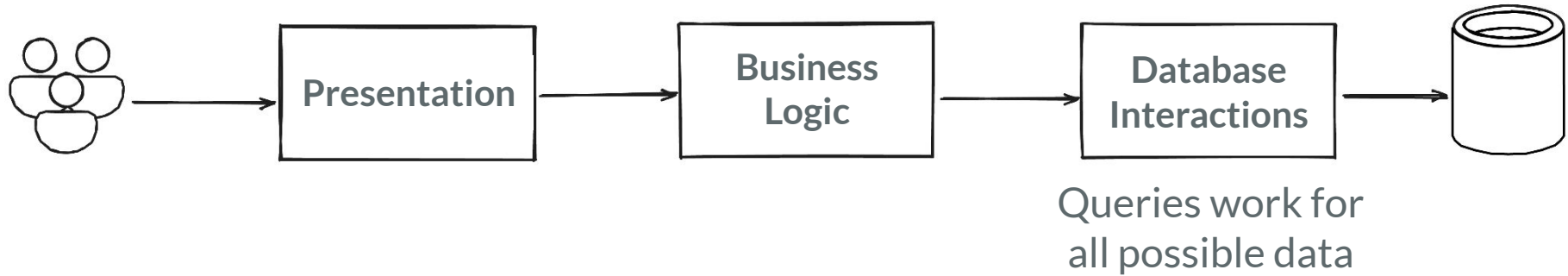
A Search For Lurking Bugs



A Search For Lurking Bugs



A Search For Lurking Bugs



Demo

github.com/mourjo/quick-meetings

Recap: Bugs Found

Recap: Bugs Found

- 5xx, non-JSON responses

Recap: Bugs Found

- 5xx, non-JSON responses
- Daylight savings validations bug

Recap: Bugs Found

- 5xx, non-JSON responses
- Daylight savings validations bug
- A SQL bug in determining overlaps

Recap: Bugs Found

- 5xx, non-JSON responses
- Daylight savings validations bug
- A SQL bug in determining overlaps
- Accepting a meeting creates an overlap

Recap: Bugs Found

- 5xx, non-JSON responses
- Daylight savings validations bug
- A SQL bug in determining overlaps
- Accepting a meeting creates an overlap
- Rejecting a meeting creates an empty meeting

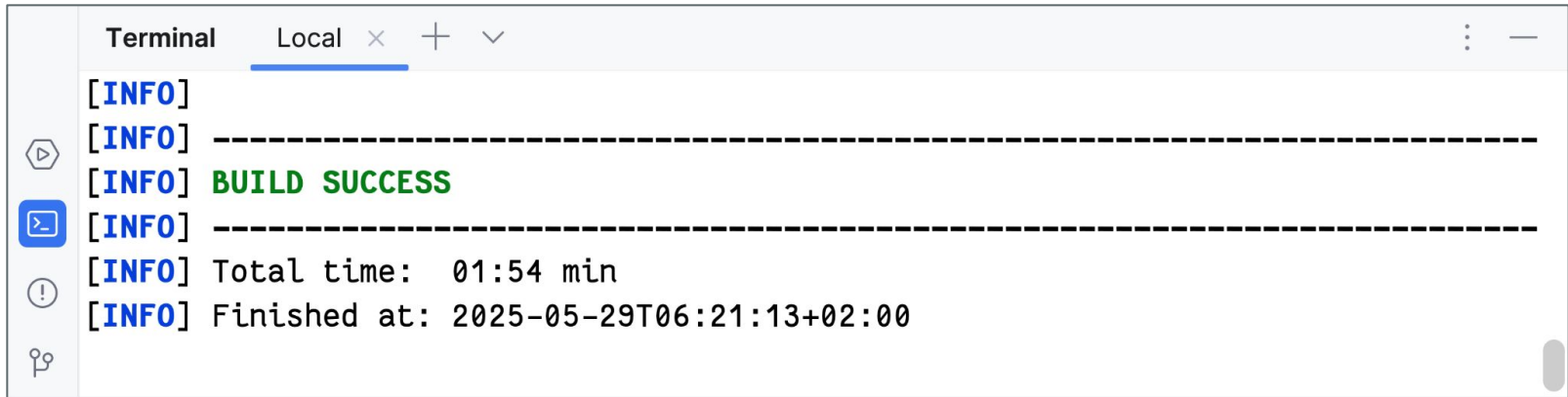
Recap: Bugs Found

- 5xx, non-JSON responses
- Daylight savings validations bug
- A SQL bug in determining overlaps
- Accepting a meeting creates an overlap
- Rejecting a meeting creates an empty meeting



Failing cases
were **not coded**
in the tests

Absence of Evidence

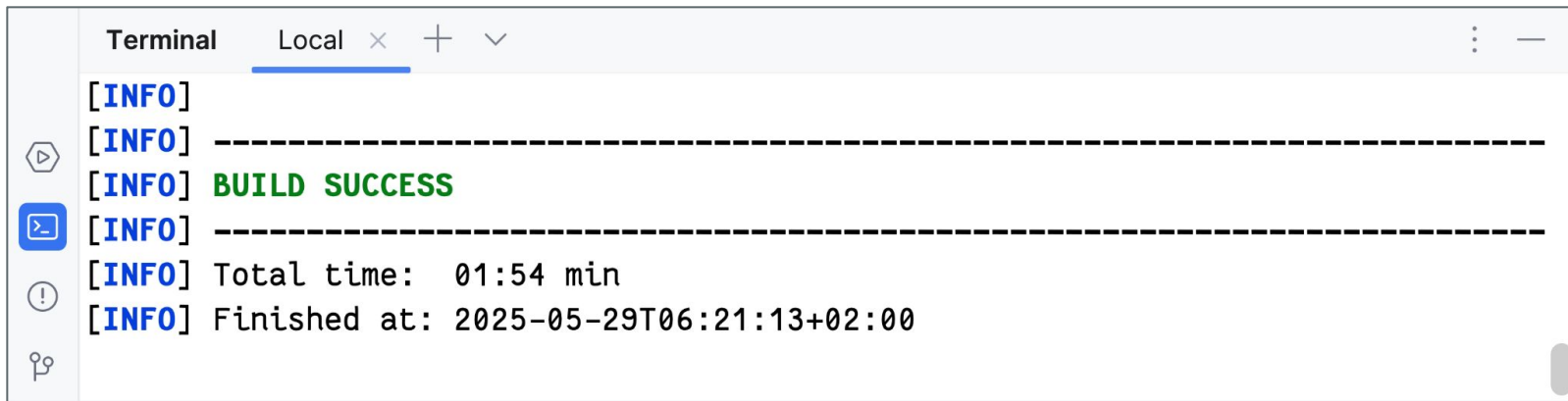


A screenshot of a terminal window with a light gray header bar. The header bar contains the word "Terminal" on the left, followed by a tab labeled "Local" with a close button (x) and a plus sign (+) to its right. On the far right of the header bar are three vertical dots and a minus sign (-). The terminal area below the header has a white background and displays several lines of text. On the left side of the terminal, there is a vertical sidebar with icons: a play button, a terminal icon (highlighted with a blue background), an exclamation mark, and a person icon. The text in the terminal is as follows: [INFO] followed by a dashed line, [INFO] BUILD SUCCESS followed by a dashed line, [INFO] Total time: 01:54 min, and [INFO] Finished at: 2025-05-29T06:21:13+02:00. A vertical scrollbar is visible on the right side of the terminal area.

```
Terminal Local x + v
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 01:54 min
[INFO] Finished at: 2025-05-29T06:21:13+02:00
```

Absence of Evidence $\neq$ Evidence of Absence

- A passing build does not imply the absence of bugs

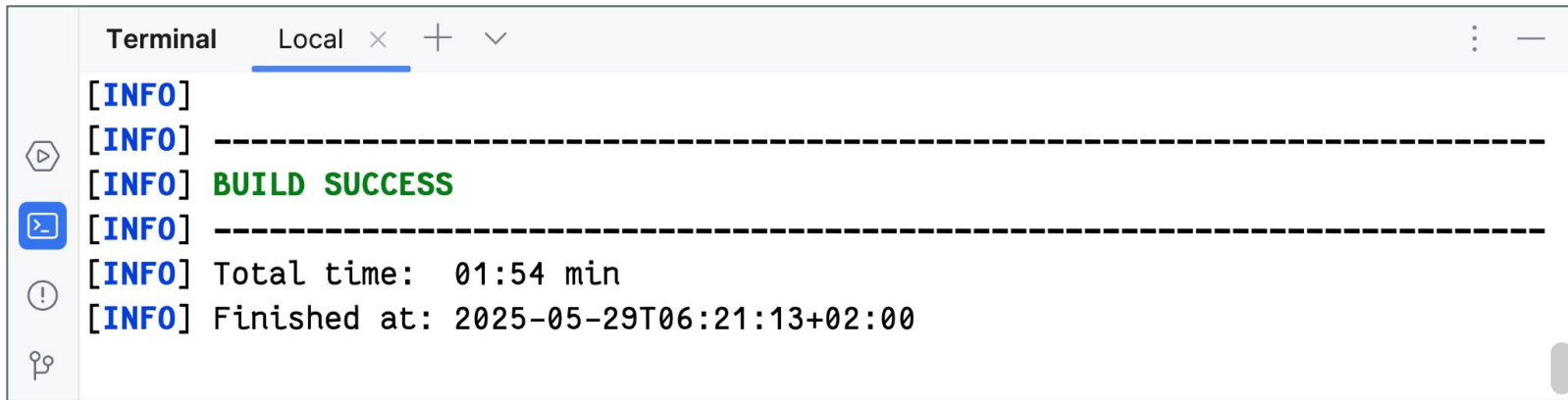


The image shows a terminal window with a light gray header bar. The header bar contains the word "Terminal" on the left, followed by a tab labeled "Local" with a close button (x) and a dropdown arrow (v). On the right side of the header bar are three vertical dots and a minus sign. The terminal area has a white background and displays several lines of text. On the left side of the terminal, there is a vertical toolbar with icons for a play button, a terminal window, an exclamation mark, and a person icon. The text in the terminal is as follows: [INFO] followed by a dashed line, [INFO] BUILD SUCCESS followed by a dashed line, [INFO] Total time: 01:54 min, and [INFO] Finished at: 2025-05-29T06:21:13+02:00. A vertical scrollbar is visible on the right side of the terminal window.

```
Terminal Local x v
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 01:54 min
[INFO] Finished at: 2025-05-29T06:21:13+02:00
```

Absence of Evidence != Evidence of Absence

- A passing build does not imply the absence of bugs
- It implies **no bug was found**

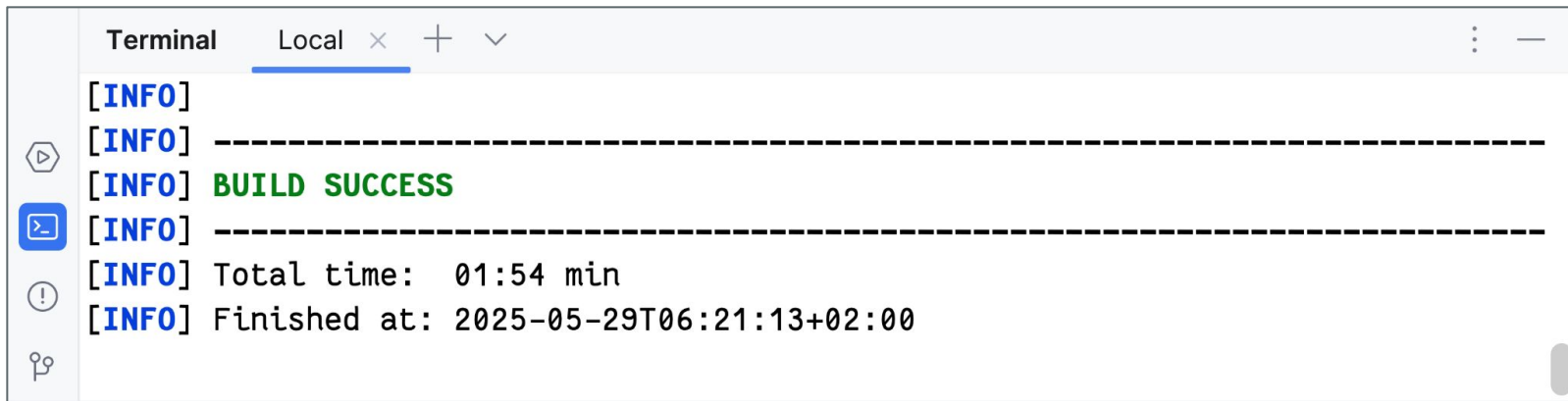


The image shows a screenshot of a terminal window with a light gray header bar. The header bar contains the word "Terminal" on the left, followed by a tab labeled "Local" with a close button (x) and a dropdown arrow (v). On the right side of the header bar are three vertical dots and a minus sign. The terminal area has a white background and displays several lines of text. On the left side of the terminal, there is a vertical sidebar with icons: a play button, a terminal icon (highlighted in blue), an exclamation mark, and a person icon. The text in the terminal is as follows:

```
[INFO]  
[INFO] -----  
[INFO] BUILD SUCCESS  
[INFO] -----  
[INFO] Total time: 01:54 min  
[INFO] Finished at: 2025-05-29T06:21:13+02:00
```


Absence of Evidence $\neq$ Evidence of Absence

- A passing build does not imply the absence of bugs
- It implies **no bug was found**
- Property based tests **searches** (and not just tests) for bugs



The image shows a screenshot of a terminal window with a light gray header bar. The header bar contains the word "Terminal" on the left, followed by a tab labeled "Local" with a close button (x) and a dropdown arrow (v). On the right side of the header bar are three vertical dots and a minus sign. The terminal area has a white background and displays several lines of text. The first line is "[INFO]". The second line is "[INFO] -----". The third line is "[INFO] BUILD SUCCESS" in green text. The fourth line is "[INFO] -----". The fifth line is "[INFO] Total time: 01:54 min". The sixth line is "[INFO] Finished at: 2025-05-29T06:21:13+02:00". On the left side of the terminal window, there is a vertical sidebar with icons: a play button, a terminal icon (highlighted with a blue square), an exclamation mark, and a person icon. A vertical scrollbar is visible on the right side of the terminal window.

```
Terminal Local x v
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 01:54 min
[INFO] Finished at: 2025-05-29T06:21:13+02:00
```

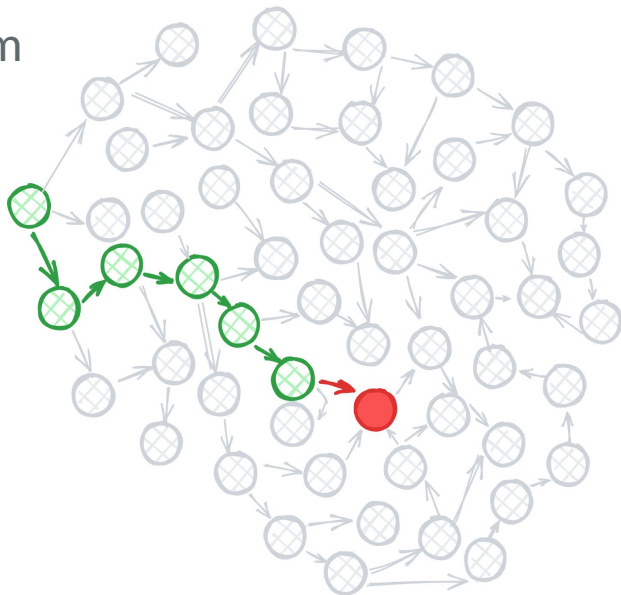
Are Your Tests Exhaustive Enough?

Are Your Tests Exhaustive Enough?

- Invariants are the **unchanging properties** of a system
 - Hard to keep validating by hand

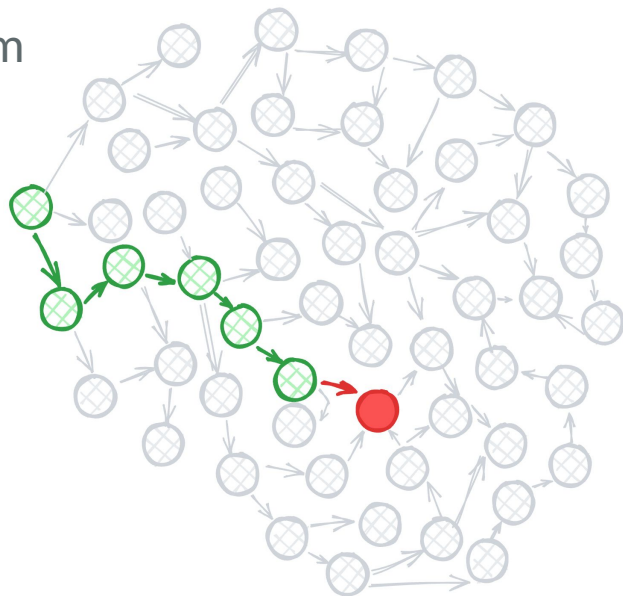
Are Your Tests Exhaustive Enough?

- Invariants are the **unchanging properties** of a system
 - Hard to keep validating by hand
- Property-based tests preserve invariants
 - **Exploration** and not enumeration
 - Effective feedback: **Minimally failing** inputs



Are Your Tests Exhaustive Enough?

- Invariants are the **unchanging properties** of a system
 - Hard to keep validating by hand
- Property-based tests preserve invariants
 - **Exploration** and not enumeration
 - Effective feedback: **Minimally failing** inputs
- Defends against **accidental quality**
 - Are your 50 test cases enough for the 1000 users?



Thank You!

Property Based Testing
to Preserve Invariants

Mourjo Sen
<https://mourjo.me>

github.com/mourjo/quick-meetings